Emp?rio da M?sica Customer Support Agent

Repository: https://github.com/Rychardsson/artefact-desafio-ia

Empório da Música Customer Support Agent

Python customer-support agent for the Empório da Música catalog, pricing, orders, policies, and human handoff.

Running

Requires Python 3.11+ and [`uv`](https://docs.astral.sh/uv/).

```
uv sync --extra dev
Copy-Item .env.example .env
# set EMPORIO_API_KEY in the environment or in .env
uv run emporio-agent
```

Equivalent bash commands:

```
uv sync --extra dev
cp .env.example .env
# set EMPORIO_API_KEY in the environment or in .env
uv run emporio-agent
```

The CLI accepts:

```
--data        CSV data directory
--session     persisted session name
--memory-dir  session JSON directory
--debug       show tool names called by the agent
```

The CLI loads `.env` automatically. To configure the provider explicitly in PowerShell:

```
$env:EMPORIO_API_KEY = "your-key"
uv run emporio-agent --debug
```

The default example configuration uses Gemini 2.5 Flash through Google's OpenAI-compatible endpoint. It was chosen because it supports tool calling, is fast enough for a support flow, and is available for this challenge with a straightforward environment-based setup. The prompt keeps the model focused on language matching, store scope, privacy, and deterministic tool use; Python remains responsible for prices, stock, policies, authentication, and deadlines.

To validate without an external provider:

```
uv run --with pytest pytest
```

Architecture

```
CLI
|-- Agent (maximum of 6 rounds)
    |-- LLMClient: Anthropic or OpenAI-compatible
    |-- ToolRegistry
    |    |-- catalog and payments
    |    |-- orders and returns
    |    |-- policies, store information, hours, and handoff
    |    `-- typed CSV data
    `-- SessionMemory (JSON)
```

The model interprets the user message and selects tools. Prices, stock, discounts, installments, shipping, authentication, and deadlines are calculated in Python.

Technical decisions

- Hybrid function calling: the domain is small and closed, so typed tools are safer than text-to-SQL.
- `Decimal`: monetary values never use `float`.

- Lexical policy retrieval: the manual is short and has stable sections, so deterministic keyword retrieval is easier to test. Embeddings would be more appropriate for a much larger manual.
- JSON memory: sufficient for local sessions and demonstrations; history is bounded by turns.
- Providers: Anthropic and OpenAI-compatible adapters allow changing providers through environment variables.

Data rules

- `coming_soon`, discontinued, and out-of-stock products are not offered.
- Only active promotions are used; promotions do not stack and the highest discount wins.
- Inconsistent descriptions are not used to override product names and specifications.
- PIX applies a 5% discount only when the items are not on promotion.
- In Campo Grande, shipping is free only above R$ 500.00; orders up to and including R$ 500.00 cost R$ 35.00.
- Order lookup requires the order number and phone, or a unique email. After three failed attempts in a session, lookup is blocked.
- Address, phone, email, and opening hours are returned by the store-information tool.
- Estimated delivery is used as a receipt-date proxy because the CSVs do not contain the actual receipt date.
- Reproducible frozen-clock scenarios use `EMPORIO_NOW=2026-03-01T10:00:00-04:00`.

Data inconsistencies are recorded in memory through `DataStore.audits`; they are not silently corrected.

Known contradictions

- The manual contains two phone numbers; the flow uses the cover phone, `(67) 3341-4444`, and does not invent a WhatsApp number.
- The manual contains email variations; the flow uses `contato@emporiodamusica.com.br`.
- The general rule mentions a R$ 100.00 minimum installment, but the more specific tiered section prevails: R$ 50.00 up to 3 installments, R$ 80.00 from 4 to 6, and R$ 100.00 from 7 to 12.
- The manual describes a 30-day defect exchange and a 90-day legal warranty; the agent explains both layers.
- The manual mentions more than 300 instruments, while the CSV contains 65 products; the CSV is the source for catalog answers.

Privacy and security

- Order data is returned only after identity validation.
- Authentication errors are intentionally generic.
- `--debug` shows tool names, not credentials or private payloads.
- Keys must remain in environment variables and must never be committed.

Tests and scenarios

Unit tests cover catalog search, promotions, payments, shipping, authentication, returns, store hours, memory, and the tool loop:

```
uv run --with pytest pytest
21 passed
```

Demonstration scenarios are listed in [`examples/scenarios.json`](examples/scenarios.json). Four conversation examples are available as Markdown transcripts: [`conversation-01-price.md`](examples/conversation-01-price.md), [`conversation-02-policies.md`](examples/conversation-02-policies.md), [`conversation-03-out-of-scope.md`](examples/conversation-03-out-of-scope.md), and [`conversation-04-store-information.md`](examples/conversation-04-store-information.md). The first three are real Gemini conversations; the fourth demonstrates the deterministic store-information tool. The consolidated machine-readable index for the real conversations is [`examples/real-conversations.json`](examples/real-conversations.json).

The English technical response is available as [`technical-answer.pdf`](technical-answer.pdf).

Known limitations

- Shipping outside the metropolitan region is not actually quoted by ZIP code.
- Holidays are not checked.
- There is no live WhatsApp, email, or support-dashboard integration.
- Handoff records exist only during the local process.
- Final response quality still depends on the configured model.

With more time

- Replace lexical policy retrieval with embeddings and evaluate retrieval quality.
- Add real shipping quotes by ZIP code, weight, and dimensions.
- Integrate human handoff with a support dashboard or messaging provider.
- Add an automated evaluation set for factuality, tool selection, language matching, and refusal behavior.

Use of AI assistants

I used GitHub Copilot with the Copilot SDK in VS Code as an implementation assistant. My workflow was to inspect the supplied PDFs and CSVs, ask the assistant to propose small implementation steps, implement and review each module, run focused tests, inspect failures, and then run the full suite. Gemini 2.5 Flash was used through the project's OpenAI-compatible adapter for the recorded customer conversations; deterministic Python tools remained the source of truth for business rules. No API key or external private data was included in the codebase.